

起源与发展

模糊测试的诞生

出发点：提升 UNIX 操作系统的可靠性

技术构想：

- 核心组件：一组用于产生随机字符的程序
- 中心思想：以随机字符串作为输入，运行操作系统组件（Utilities），观察是否崩溃
- 最终结果：保留能够产生崩溃的字符串输入，分析崩溃的类型，对崩溃进行分类

结果：在 22 个（总 90 个）组件程序（Utility Program）上触发崩溃

概念与框架

初始构想

三要素：一个（套）工具、一个目标、一个循环

- 工具：模糊器（Fuzzer）
- 目标：待测程序（PUT）
- 循环：执行程序 $\rightleftharpoons$ 崩溃分派（Crash Triage）

相关术语

模糊（Fuzzing）与模糊测试（Fuzz Testing）

模糊（Fuzzing）与模糊测试（Fuzz Testing）

模糊是指使用从输入空间（模糊输入空间）采样得到的输入来执行待测程序（PUT, Program Under Test）的过程。该模糊输入空间代表着测试人员针对待测程序定义的预期输入。

模糊测试是一种应用模糊（过程）来验证待测程序是否违反正确性策略（Correctness Policy）的测试技术。

在文献中一般可互换。

模糊器（Fuzzer）

模糊器是一个（组）用于实现模糊测试的程序 $\rightarrow$ 核心组件

模糊运动（Fuzzing Campaign）

模糊运动是指一个模糊器按照一组特定的正确性政策在一个给定待测程序上的一次具体的执行。

$\rightarrow \langle \text{fuzzer}, \text{program} \rangle$

缺陷预言（Bug Oracle）

一个用于确定一次给定执行是否违反具体正确性策略的程序，通常作为模糊器的一部分出现。

模糊配置 (Fuzz Configuration)

一组控制和描述模糊（测试）算法的数据和约束

测试输入 (Test Input)、测试用例 (Test Case) 与种子输入 (Seed Input)

- 测试输入是一组用于驱动待测程序执行的数据
- 测试用例是一组用于确定应用软件或软件系统是否能够正确工作的条件或变量 → 输入+ 逻辑（调用序列） + 预言
- 种子输入是一个或一组在模糊测试过程中为输入生成（Input Generation）提供基准的测试输入，简称种子（Seed）

模糊测试框架



家族与分类

根据基础 Fuzzer 划分家族：

- AFL 家族 (C/C++)：AFL、AFLFast、AFLSmart、AFLNet、AFLGo、AFLIoT、FairFuzz、Mopt、Neuzz
- LibFuzzer 家族 (C/C++)：LibFuzzer、Entropic
- JQF 家族 (Java)：JQF、BeDivFuzz、CONFETTI
- 其他 (Rust、Python 等)：Angora、DeepXplore

根据组件核心或技术贡献进行分类：

- 按照采用的运行时信息：黑盒、灰盒、白盒
- 按照输入生成的策略：Mutation-based, Generation-based
- 按照引导过程：Search-based（一些启发式算法），Gradientbased（梯度下降）
- 按照测试的目的：定向、非定向、某一类缺陷
- 按照应用领域：网络协议、Compiler、DNN、IoT、内核
- 按照优化角度：种子调度、变异策略、能量调度、过程建模

按照运行时信息分类

黑盒 (Blackbox) 模糊测试

特点：不监控执行过程，也不使用执行过程中产生的任何信息，仅从输入和输出端入手优化模糊测试

引导方式：利用输入格式或输出状态引导测试执行

优缺点：效率高，但引导的有效性上面有所欠缺

代表性工作：KIF、IoTfuzzer、CodeAlchemist

白盒 (Whitebox) 模糊测试

特点：使用混合执行、污点分析 (Taint Analysis) 等比较昂贵的白盒分析技术优化模糊测试过程

引导方式：利用详细的程序分析结果引导测试执行

优缺点：反馈更加有效，但是效率不高、适配性较差

代表性工作：Driller2、QSYM3、CONFETTI4

灰盒 (Greybox) 模糊测试

Coverage-based Greybox Fuzzing, CGF

特点：采用轻量级插装对程序进行监控，在执行过程中收集各类信息，如分支覆盖、线程执行、堆栈状态等

引导方式：利用收集到的执行信息（内部状态）引导测试执行

代表性工作：AFL、AFLGo、EcoFuzz、Zest、BeDivFuzz

 CGF 伪代码

按照生成策略分类

按照输入生成的策略：

- Input Generator、Mutation Strategies
- Mutation-based：基于随机或启发式变异策略
- Generation-based：基于一定的文法规则/结构信息

Mutation-based：基于随机变异或启发式变异策略

本质：将种子输入转换为比特串 (Bits)，对比特串进行变换

优点：可拓展性强，易于泛化，理论上可用于任意输入（图片、文本、音视频）

缺点：容易破坏输入的结构、产生无效无效输入 (Invalid Input)；生成的大部分输入都无法通过语义检查 (Semantic Checking)，很难探测深层次的程序状态和程序元素 (Program Elements)

Mutation-based：AFL

AFL 变异算子

- bitflip L/S：以 S 为增量，每次翻转 L 位
- arith L/8：加/减长度为 L 的小整数
- interest L/8：翻转“有趣”字节位
- havoc：对输入进行大肆破坏
- splice：随机拼接两个种子输入

 Fuzzing a Picture

Mutation-based: SLF (ICSE'19)

思想：分析程序中的语义检查、识别比特串中与影响语义检查的域（Field）、根据两者之间的关系制定变异策略

 一个合法 Zip 文件的比特串

 字节分组	 类型推断与关系建立
--	---



Generation-based: 基于一定的文法规则/结构信息

本质：利用给定的、或者挖掘/学习得到的文法规则，来构建能够通过（语法）检查的结构化输入

优点：容易生成合法、有效输入（Valid Inputs），适用于对输入结构性要求较高的场景，如针对解析器（Parser）、解释器（Interpreter）以及编译器（Compiler）的测试

缺点：需要人工赋予一定的领域知识。在没有人工指定文法的情况下，很难得到完整的文法规则

Generation-based: Inputs from Hell (TSE'19)

思想：挖掘已有的测试输入，得到现有的测试输入分布。根据该分布进行输入生成以得到预期的测试输入

- 表示输入的分布：概率文法（Probabilistic Grammar）
- 预期输入：
 - 与概率文法相同：生成与文法表示的输入相似的测试输入
 - 与概率文法相反：生成与文法表示的输入互补的测试输入

按照引导方式分类

- Test Execution、Output Analysis
- Search-based：将测试转化为搜索问题，以代码覆盖率作为指示器、以启发式算法（类遗传算法）为核心，将测试导向更高覆盖的方向 → CGF
- Gradient-based：将测试转化为优化问题，以最大化缺陷发掘输入为目标构建目标函数，迭代求最优解 → 退而求覆盖率
- 模糊测试中的马太效应（Matthew effect）：好的种子输入能够演化产生品质良好的子代测试输入

基于搜索的模糊测试

Search-based: Fuzzing as Search Problem

- 核心：将模糊测试过程建模为搜索问题，根据模型构造启发式算法（Heuristic）解决问题
- 启发式：遗传算法-AFL、马尔科夫链-AFLFast、信息熵-Entropic、多臂老虎机问题-EcoFuzz

Search-based: AFL, 基于插装的遗传算法

基于梯度的模糊测试

Gradient-based: Fuzzing as Optimization Problem

- 核心：将模糊测试过程建模为优化问题，问题的目标是最大化缺陷发掘数量，利用梯度下降算法持续求解最优解
- 目标退阶：缺陷离散分布且无法预知 → 替换为代码覆盖
- 应用 DL 技术-Neuzz & MTFuzz；利用梯度下降取代符号执行的约束求解过程-Angora

Gradient-based: Neuzz1

核心思路：将模糊测试建模为无约束优化问题，利用梯度下降算法优化模糊测试过程

动机：Gradient Descend > Evolutionary Algorithm

 梯度下降和演进算法在高阶优化问题上的表现

挑战：

- 如何计算反馈 → 如何计算梯度
- 优化目标（缺陷发现数目或覆盖率）是不连续的，无法直接适配梯度下降算法 → 引入程序平滑（Program Smoothing）技术

程序平滑（Neural Program Smoothing）：消除目标函数中的不连续性

- 黑盒程序平滑：简单易用，但是会产生较大的近似错误
- 白盒程序平滑：依赖符号执行（Symbolic Execution）和抽象解释技术（Abstract），会产生较大的额外开销
- 神经程序平滑（Neural Program Smoothing）：利用 NN 模型模拟程序行为

 Neuzz 神经程序平滑

按照测试目的分类

非定向模糊测试：Wider and Deeper

- 目标：验证程序的正确性，检测程序中潜在的缺陷

定向模糊测试：Directed and Targeted

- 目标：针对程序中的某个目标位置进行快而有效的测试
- 场景：缺陷复现、补丁检验、静态分析报告验证
- 分类：白盒、灰盒

定向灰盒模糊测试

AFLGo：定向灰盒模糊测试

- 基本思路：Distance + Annealing-based Scheduling，为更靠近目标位置的种子分配更多的能量

iSE 模糊测试

- Predoo：基于模糊测试的深度学习算子精度测试，ISSTA'21
- Duo：基于差分测试的深度学习算子精度测试，TR'21
- Gandalf：基于生成的深度学习框架模糊测试，TBD
- QATest：问答系统模糊测试框架，ASE'22
- 探索如何利用变异测试优化模糊测试，Internetwork'22

- AFLFun: 利用函数重要度优化灰盒模糊测试, TBD

变异测试+模糊测试

Investigating Coverage Guided Fuzzing with Mutation Testing, Internetwork'22

动机 1: 现有 CGF 技术存在不足:

- 现有的(灰盒)覆盖率引导的模糊测试往往只关注如何提升代码覆盖率
- 覆盖更多的代码并不意味着能触发更多的 Bug → PIE 模型
- 仅关注分支覆盖可能使得 Fuzzer 错过能够发现缺陷的输入, 从而会损害技术的有效性

动机 2: 变异测试的优势:

- 变异测试/分析技术能够从缺陷的角度出发对测试输入进行评估, 能够帮助 Fuzzer 识别具有缺陷暴露 (Fault-revealing) 能力的测试输入
- 借助马太效应我们可以在模糊测试过程中演化生成具有更强缺陷暴露能力的测试输入

挑战: 变异测试和模糊测试存在天然的冲突

- 问题 1: 如何保证执行的效率? → 变异体筛选+ 并行执行
- 问题 2: 如何判断变异杀死? → 差分测试

差分测试: 为变异测试提供测试预言

函数重要度+模糊测试

AFLFun: 利用函数重要度增强灰盒模糊测试

- 动机: 待测函数亦有差距! 不同待测函数的重要度不同
- 挑战: 量化函数重要度, 为执行重要函数的种子分配更多资源

 Mjs 项目中存在的调用关系不均衡现象

 重要度感知的函数调用图

AFLFun: 计算重要度 (FS, Function Significance)

- 动静态结合: 采用静态函数距离和动态调用概率描述函数关系
- 中心度分析: 采用 PageRank 算法推动重要度流向处于中心函数

AFLFun: 能量调度